

BUBT Banking System

C++ Console Application — Project Report

Course	Data Structures & Algorithms Lab
Institution	Bangladesh University of Business and Technology (BUBT)
Department	EEE / CSE
Language	C++
Submission Date	2025

Team Members

#	Name	Contribution
1	Md. Jarif Chowdhury	Hash Table — Account creation & search
2	Md. Abdullah Al Noman	Queue & Stack — Service line & history
3	Md. Ahosan Habib Tanjim	Main menu & integration

1. Introduction

The BUBT Banking System is a console-based application developed in C++ that simulates core banking operations. The project is designed to demonstrate the practical application of fundamental data structures — Hash Table, Queue, and Stack — in a real-world banking scenario. The system allows users to create accounts, manage a service queue, and track action history.

2. Objectives

- Implement a Hash Table for efficient account storage and retrieval.
- Use a Queue data structure to manage customer service lines.
- Apply a Stack to maintain a history of banking actions.
- Integrate all data structures into a unified, menu-driven application.
- Demonstrate practical use of C++ programming concepts.

3. Data Structures Used

Data Structure	Purpose	Implementation
Hash Table	Account storage & lookup	Array of size 100 with linear probing
Queue	Customer service line	Array-based queue with front/rear pointers
Stack	Action history tracking	Array-based stack with top pointer

4. System Features

#	Feature	Description
1	Create Account	Creates a new bank account using Hash Table with linear probing for collision handling
2	Join Queue	Adds a customer ID to the service queue (Enqueue operation)
3	Leave Queue	Removes a customer from any position in the queue
4	Serve Next	Removes and serves the front customer (Dequeue operation)
5	Search Account	Searches for an account by ID using Hash Table lookup
6	View Last Action	Displays the most recent action from the history Stack

5. Implementation Details

5.1 Hash Table (Member 1)

The account storage system uses a Hash Table of size 100. Each account is stored using the formula: $\text{index} = \text{account_id} \% 100$. In case of collision, linear probing is applied — the system checks the next available slot sequentially. Each account stores an ID, name, and balance.

5.2 Queue — Service Line (Member 2)

An array-based Queue manages the customer service line. The front pointer tracks the next customer to be served (Dequeue), and the rear pointer tracks the end of the queue (Enqueue). A middle-removal feature allows any customer to leave the line, with elements shifted to maintain queue order.

5.3 Stack — Action History (Member 2)

A Stack records all performed actions as strings. Each new action is pushed onto the stack (push operation). The 'View Last Action' feature retrieves the top element without removing it, showing the most recent banking activity along with the total count.

5.4 Main Menu & Integration (Member 3)

The main() function provides a loop-based menu system with 7 options. It integrates all three data structures and handles invalid input gracefully using `cin.clear()` and `getline()`. Every user action is automatically logged to the history Stack.

6. How to Run

Compile and run using g++ from the terminal:

```
g++ bank_project.cpp -o bank
```

```
./bank
```

Alternatively, open `bank_project.cpp` in Code::Blocks or Dev-C++ and click Build & Run.

7. Sample Output

```
--- BUBT BANKING SYSTEM (Queue & Hash) ---
1. Create Account (Hash Table)
2. Join Line (Queue Enqueue)
3. Leave Line (Queue Removal)
4. Service Next (Queue Dequeue)
5. Search Account (Hash Table Search)
6. View Last Action (Stack Top)
7. Exit
Enter Choice: 1
ID: 101
Name: Noman
Deposit: 5000
```

[Success] Account created for Noman

8. Conclusion

The BUBT Banking System successfully demonstrates the practical integration of three fundamental data structures — Hash Table, Queue, and Stack — within a single C++ application. Each team member contributed a distinct module, resulting in a clean, modular codebase. The project reinforces the importance of choosing appropriate data structures for real-world problems and provides a strong foundation for future software development projects.